

Data Structure

Minor_Heat

目录

- 碎碎念
- 正文
 - 并查集
 - 堆
- 作业

“ 数据结构是在计算机中存储、组织数据的方式。小到变量、数组，大到线段树、平衡树，都是数据结构。 —— oi-wiki ”

“ P3919 【模板】可持久化线段树 1（可持久化数组） ——luogu ”

那么数组显然是入门数据结构吧，那么作为数组拓展，可持久化线段树一定是数据结构入门的东西吧，那我再数据结构基础讲基于可持久化线段树的可持久化并查集问题不大吧 (((

算了我还是友善一点吧（快感谢我快感谢我我才不会说是应为我没写好

由于作者语文比较菜，本课件中会大段引用 oi-wiki 中的内容，之后课件中出现在引号中的默认都是 oi-wiki 内容（但是不在引号中不代表不是 oi-wiki），敬请谅解。

并查集

“ 并查集是一种用于管理元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素。

顾名思义，并查集支持两种操作：

- 合并 (Unite)：合并两个元素所属集合（合并对应的树）。
- 查询 (Find)：查询某个元素所属集合（查询对应的树的根节点），这可以用于判断两个元素是否属于同一集合。

”

上面这段话写成代码就是这样：

```
int find(int u){if(f[u]==u)return u;return f[u];}
void merge(int u,int v){f[find(u)]=v;}

while(q--){
    cin>>op>>x>>y;
    if(op==1)merge(x,y);
    else cout<<(find(u)==find(v)?"Y\n":"N\n");
}
```

以上就是一个最基础的，不加任何优化的并查集。可以想到，如果我们这样去完成上述操作，单次操作就可以被卡到 $O(n)$ 。那么如何提高效率呢？这里介绍两种常用优化。

路径压缩

试想我们每一次找到所在集合的根部都需要一步一步向父亲跳出一条链，但实际上我们需要的只有这个节点的最上面的那个祖先。也就是说，完全可以直接将该节点指向其祖先节点来避免浪费这些查询的时间。这个优化虽然改变了并查集的形态使其不利于回溯倒过去时刻，但对时间的优化效果非常显著，并且一般做题时我们也不需要对比查集进行回溯，所以没有影响。

代码：

```
int find(int u){if(f[u]==u)return u;return f[u]=find(f[u]);}
```

按秩合并

另一种思想是通过变幻连接方式使树高本身保持一个比较小的值。这样不用路径压缩也可以大大节省查找根节点的时间。

对于连接方式，我们一般选取将树高较小的并查集接到较大的上，或者将树的大小较小的并查集接到较大的上，两者时间复杂度相近，但是笔者认为按树高合并写起来太麻烦了，所以就用按树的大小了。

代码：

```
void merge(int u, int v) { if (sz[u=find(u)] < sz[v=find(v)]) swap(u, v); sz[u] += sz[v]; f[v] = u; }
```

并查集的复杂度

单独使用路径压缩或者安置合并的代码都是 $O(q \log n)$ 的，但是如果将两者结合起来，可以得到 $O(q\alpha(n))$ 的算法，其中 $\alpha(n)$ 是个非常小的数，计算时可以把它当成常数。

然而一般的题目多加一个 $\log n$ 的时间复杂度貌似不影响能否通过题目，而且笔者认为按秩合并实在是太麻烦啦，所以后面提供的参考代码大多是只加了路径压缩优化的，轻喷谢谢 qwq。

P3367 【模板】 并查集

现有一个并查集，你需要完成合并和查询操作。

输入：第一行两个整数 N, M ，表示 N 个元素， M 个操作。接下来 M 行每行三个整数 op_i, x_i, y_i ， $op_i = 1$ 表示合并 x_i, y_i 所在集合， $op = 2$ 表示询问 x_i, y_i 是否在同一集合内。

输出：对于每一个 $op_i = 2$ ，输出一行表示结果，其中是的输出 **Y**，否则输出 **N**。

数据范围： $1 \leq N \leq 2 \times 10^5$ ， $1 \leq M \leq 10^6$

代码实现

```
#include <bits/stdc++.h>
using namespace std;
int n,m,op,x,y,f[10005];
int find(int u){if(f[u]==u)return u;return (f[u]=find(f[u]));}
void merge(int u,int v){u=find(u);v=find(v);if(u==v)return ;f[u]=v;}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>m;for(int i=1;i<=n;i++)f[i]=i;
    while(m--)
    {
        cin>>op>>x>>y;
        if(op==1)merge(x,y);
        if(op==2)cout<<(find(x)==find(y)?"Y\n":"N\n");
    }
    return 0;
}
```

P3958[NOIP2017提高组]奶酪

一个空间中有若干个球，你只能在球中行走，问是否能从上面走到下面。

输入:: 第一行一个正整数 T ，表示数据组数。每组数据第一行三个整数 n, h, r ，接下来 n 行每行三个整数 x, y, z 表示球心坐标。

输出: 对于每组数据输出一行，如果能走到上表面则输出 **Yes**，否则输出 **No**。

数据范围: $1 \leq T \leq 20, 1 \leq n \leq 1000, 1 \leq h, r, |x|, |y|, |z| \leq 10^9$

- 将洞视为点，上下表面再设一个点
- 根据半径判断任意两点是否联通
- 做并查集
- 时间复杂度 $O(n^2)$

$P1196[NOI2002]$ 银河英雄传说

给定 T 次操作，若干个队列，每次要不将两个队列合并，要不查询两数是否在同一队列，若在，还需给出两数之间差了多少位置。

输入：第一行一个整数 T ，然后输入 T 行表示操作， $M\ i\ j$ 表示将战舰 i 一列接到战舰 j 一列后面， $C\ i\ j$ 表示询问两战舰是否在同一列，若在，中间隔了多少战舰。

输出：对于每个 $C\ i\ j$ ，输出一行一个数字。若不在同一列输出 -1 。

- 首先注意到这题的合并、查询操作很像并查集
- 并查集的两点间战舰数就是路径长度差，但是这一性质在路径压缩时被打破了
- 一列中的战舰作为一个整体，其相对位置不再发生改变，所以可以多存一个变量表示节点到其指向节点的路径长度，压缩时维护一下就好

$P1197[JSOI2008]$ 星球大战

n 个点， m 条边，初始保证图联通。 k 次操作，每次删一个点，每次操作后输出连通块数。

输入：第一行两个整数 n, m ，接下来 m 行每行两个整数 u, v 表示一条边。第 $m + 2$ 行一个整数 k ，接下来 k 行每行一个整数，表示按顺序被摧毁的点的编号（0 到 $n - 1$ ）。

输出： $k + 1$ 行，每行一个整数表示连通块数。

数据范围： $1 \leq n \leq 2 \times 10^5$ ， $1 \leq m \leq 2 \times 10^5$ ， $1 \leq k \leq n$ 。

- 删除比较困难，考虑反过来做
- 题目变为每次点连边，询问连通块数
- 每次连边判断连通块数是否减少即可。

堆

“堆是一棵树，其每个节点都有一个键值，且每个节点的键值都大于等于/小于等于其父亲的键值。

每个节点的键值都大于等于其父亲键值的堆叫做小根堆，否则叫做大根堆。STL 中的 `priority_queue` 其实就是一个大根堆。

（小根）堆主要支持的操作有：插入一个数、查询最小值、删除最小值、合并两个堆、减小一个元素的值。

习惯上，不加以限定提到「堆」时往往都指二叉堆。

”

二叉堆

本文默认讨论的是大根堆。

二叉堆形式上是一颗完全二叉树，而其中每个节点的父节点都比他要大，也就是说，堆顶就是这个堆中的最大值。

在插入与删除元素的过程中，我们一定会遇到堆的结构被破坏的情况，这时就需要调整。调整的方法就是 up 操作和 down 操作。

上移

当插入一个元素时，我们选择先将其放在堆最后一层的最左边的一个空位（如果最后一层已满则新增一层）。这时堆的性质就被破坏了，原因是这个节点可能会比上面的节点要大。于是我们进行上移操作。每次比较这个节点与他的父节点，如果这个节点更大，将两者交换位置，否则退出循环。

下移

下移是另一种调整方式，听名字就可以知道，它是与上移相对应的一种方案，具体的用处是：当我们想要删除一个节点时，可以先把它与最后面的一个节点交换位置直接删除，让后将被换上来的节点不断向下移动来保持堆的性质。移动时要注意每次找到它的两个儿子节点中较大的那个。

移动的正确性证明与复杂度

首先可以发现，无论上移还是下移，移动都仅仅影响了一条链，所以这条链以外的部分的堆的性质一定没有被破坏。而链上，相邻两个节点间的大小关系也没有被改变，所以只要关注链上的每个节点与它儿子之间的大小关系了。

在上移时，相当于我们把一条链向下扯了一格，并且将新的这个更大的元素放在了空出来的位置。这样调整后，每个位置上的节点都不会小于原来的节点，所以“比两个儿子大”这一性质不会被破坏。而下移时，被换上去的那个节点本身就比较另一个节点和当前根节点要大，所以也满足了堆的性质。故无论上移下移，调整结束后都能做到满足堆的性质。

data structure 由于仅仅操作了一条链，时间复杂度就是树高，也就是 $O(\log n)$ 。

P3378 【模板】 堆

给定一个数列，初始为空，你需要支持几种操作。

输入：第一行一个整数 N ，表示 N 个操作。接下来 N 行输入。每行先给出一个 op_i ， $op = 1$ 表示插入一个数 x ， $op = 2$ 表示询问数列最小值， $op = 3$ 表示删除数列最小值（多个最小值仅删除 1 个）。

输出：对于每个 $op = 2$ ，输出一行表示结果。

数据范围： $1 \leq N \leq 10^6$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
int n, op, x, q[1000005], l;
void up(int p){while(p!=1&&q[p]<q[p>>1]){swap(q[p], q[p>>1]);p>>=1;}}
void down(int p){while((p<=1)<=1){if(p<1&&q[p]>q[p+1])p++;if(q[p]>=q[p>>1])break;swap(q[p], q[p>>1]);}}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n;
    while(n-->0)
    {
        cin>>op;
        if(op==1){cin>>x;q[++l]=x;up(l);}
        if(op==2)cout<<q[l]<<"\n";
        if(op==3){q[l]=q[l--];down(l);}
    }
    return 0;
}
```

*P*1801黑匣子

看原题吧，有点写不下qwq

- 跟模板题差不多，按题意模拟即可

*P1168*中位数

给定一个长度为 N 的非负整数序列 A ，对于前奇数项求中位数。

输入：第一行一个正整数 N ，第二行 N 个正整数 $A_1 \dots A_N$ 。

输出：共 $\lfloor \frac{N+1}{2} \rfloor$ 行，第 i 行为 $A_1 \dots A_{2i-1}$ 的中位数。

数据范围： $1 \leq N \leq 10^5$ ， $0 \leq A_i \leq 10^9$ 。

- 维护一个大根堆，一个小根堆，其中大根堆放前 $\frac{N}{2}$ 小的数，小根堆放剩下的数。
- 插入一个值时，先放进大根堆，然后根据两个堆的大小调整一下，注意如果大根堆堆顶比小根堆大了，就交换两个堆顶。
- 每次输出堆顶即可。
- 这是一个对顶堆模板，请务必记住这种做法。

*P1631*序列合并

有两个长度都是 N 的序列 A 和 B ，在 A 和 B 中各取一个数相加可以得到 N^2 个和。求这些和中最小的 N 个。

输入：第一行一个整数 N ；第二行 N 个整数 A_i ；第三行 N 个整数 B_i 。

输出：一行 N 个整数，从小到大输出这 N 个最小的和。

数据范围： $1 \leq N \leq 10^5$ ， $1 \leq A_i, B_i \leq 10^9$ 。数据保证 A 和 B 均已升序排列。

- 将问题转化为 N 个由 $A_i + B_j$ ($j = 1..N$) 的数组的合并，且由于 B 升序，数组内部也是升序的。
- 维护前 N 小的值。
- 正确性：由于每个链表内部有序，且堆每次取出全局最小值，保证了取出的 N 个和就是最小的 N 个。
- 复杂度： $O(N \log N)$ ，堆中最多同时存在 N 个元素。

作业

<https://vjudge.net/contest/778280>

GL&HF